

Python Word Guessing Game

This documentation explains the **Guess the Word** game, which is a simple Python-based word-guessing game. It is designed to help beginners learn **Python concepts interactively** while having fun.

Overview

The **Guess the Word** game is an engaging Python-based program that allows players to guess a hidden word. The game provides a **hint** for each word, making it an exciting way to reinforce Python terminology.

Features

- **Random word selection from a predefined list**
- **Hints to guide players**
- **Turn-based guessing system**
- **Winning and game-over scenarios**

The project is built using:

- **Python's random module** for word selection
 - **Loops and conditions** for game logic
 - **String operations** for word display
-

How the Game Works

1. The game welcomes the user and asks for their **name**.
 2. It randomly selects a Python-related **word** from a predefined list.
 3. The **hint** (basic definition) of the selected word is displayed.
 4. The player must **guess letters** of the word one by one.
 5. If the guessed letter is correct, it appears in its position.
-

Step 1: Setting Up the Project

First, create a new project folder and add the following files:

```
/L0-Guess-The-Word  
| — Guess-The-Word.py (Main Python script)
```

This file will contain the Python logic for the game.

Step 2: Writing the Python Code

Open **Guess-The-Word.py** and add the following code:

(a) Game Introduction & Player Name Input

```
import random  
  
# Welcome message  
print("Welcome to the Word Guessing Game!")  
  
# Getting player's name  
name = input("What is your name? ")  
print(f"Good Luck, {name}! Let's begin!")
```

- Greets the player and prompts them for their name.
 - Personalizes the experience by displaying the name.
-

(b) Word Selection with Hints

```
# Word list with hints
words_with_hints = {
    'variable': "A container that holds data in Python.",
    'function': "A reusable block of code that performs a task.",
    'loop': "A control structure used to repeat a block of code.",
    'integer': "A data type used to store whole numbers.",
    'string': "A data type used to store text in Python.",
    'boolean': "A data type that holds True or False values.",
    'list': "A collection that stores multiple values in a single variable.",
    'tuple': "An immutable collection of ordered values.",
    'dictionary': "A data structure that stores key-value pairs.",
    'condition': "A statement that checks whether something is True or False.",
    'iteration': "The process of repeatedly executing a block of code.",
    'recursion': "A function that calls itself to solve a problem."
}

# Selecting a random word and its hint
word, hint = random.choice(list(words_with_hints.items()))

# Displaying hint to help the player
print("\nHint:", hint)
```

- Defines a **dictionary of words with hints**.
 - Uses `random.choice()` to select a **random word** and its corresponding **hint**.
 - Displays the **hint** to assist the player.
-

(c) Game Variables

```
# Variables for game logic
guesses = ""
turns = 10
```

- `guesses` stores all **user inputs**.

- turns represents the **number of attempts** before the game ends.
-

(d) Game Loop (Guessing & Display Logic)

```
while turns > 0:

    failed = 0 # Counter for unguessed characters
    display_word = ""

    for char in word:
        if char in guesses:
            display_word += char + " "
        else:
            display_word += "_ "
            failed += 1

    print("\nWord: ", display_word)
```

- **Loops** until the player runs out of attempts.
 - Replaces unguessed letters with _ and displays the **partially revealed word**.
 - Counts the number of **remaining letters to be guessed**.
-

(e) Checking for Win Condition

```
if failed == 0:
    print("\n Congratulations! You guessed the word:", word)
    break
```

- If **all letters are guessed**, the player **wins** and the game **ends**.
-

(f) Handling Player Input

```
guess = input("\nGuess a character: ").lower()

if guess in guesses:
    print(" You already guessed that letter. Try another one.")
    continue

guesses += guess
```

- Takes the **user's guess** and ensures it is in **lowercase** for consistency.
 - Checks if the **letter was already guessed** and prompts the player to try another.
-

(g) Incorrect Guess Handling

```
if guess not in word:
    turns -= 1
    print("\n Wrong guess!")
    print(f"You have {turns} turns left.")

if turns == 0:
    print("\n Game Over! The correct word was:", word)
```

- If the **letter is not in the word**, it reduces the **number of turns**.
 - If turns == 0, the **game ends** and displays the correct word.
-

Key Features

Engaging & Educational – Helps students learn Python terms in a fun way.

Interactive Gameplay – Provides hints and dynamically updates the word.

Turn-Based System – Encourages strategic guessing.

Randomized Words – Each game offers a different challenge.

How to Run the Project

1. **Install Python** (if not installed).
2. **Save the script** as `guess_the_word.py`.
`python guess_the_word.py`
3. **Follow the on-screen instructions** to play the game.

Conclusion

This **Guess the Word** game provides an engaging way to learn **Python programming terms** while testing logical thinking. It is an excellent **beginner-friendly** project that reinforces concepts such as **loops, conditions, dictionaries, and user input handling**.